

Orientation to Computing

Comprehensive University Study Guide

CSE111 · Semester 1 Study Guide

Languages ▪ Fundamentals ▪ Hardware ▪ Number Systems ▪ Git ▪ AI Tools

Curriculum Features:

- Complete coverage of all 6 Units of the official LPU CSE111 syllabus.
- **Unit 1 & 2:** Computer Languages, Translation Pipeline, Von Neumann Architecture, Generations 1–5.
- **Unit 3 & 4:** CPU vs. GPU, Memory Hierarchy, DRAM/SRAM, SSD/HDD, Binary/Octal/Hexadecimal Systems.
- **Unit 5 & 6:** Git Distributed Version Control, GitHub Workflows, Generative AI Tools & Prompt Engineering.

Editorial Team / Publisher:

LPU Engineering Open Publishing Project

<https://github.com/LPU-Engineering-Open>

CSE111: Fundamentals of Computing & Information Technology

An Open-Source Educational Publication

Published by: LPU Engineering Open Initiative

Release Edition: 2026 Edition (Phase 1 Base Architecture)

Primary Source Repository: <https://github.com/LPU-Engineering-Open>

Licensing:

© 2026 The Contributors of LPU Engineering Open.

This manuscript, diagrams, and educational text are distributed under the terms of the **Creative Commons Attribution-NonCommercial-ShareAlike 4.0 International (CC BY-NC-SA 4.0)** License. To view a copy of this license, visit <https://creativecommons.org/licenses/by-nc-sa/4.0/>.

The accompanying LaTeX macros, shell scripts, and build files are distributed under the terms of the **MIT License**.

Suggested Citation:

LPU Engineering Open Contributors, *CSE111: Fundamentals of Computing & Information Technology — Study & Practice Guide*, 2026 ed. Open Educational Resources.

Important Disclaimer & Institutional Non-Affiliation

1. Independent Student-Created Resource

This book is an independent, non-profit, student-led open educational resource designed to aid students in learning computing fundamentals and preparing for university engineering examinations.

2. Non-Affiliation Notice

LPU Engineering Open is NOT affiliated with, sponsored by, authorized by, or endorsed by Lovely Professional University (LPU). Lovely Professional University is not responsible for the contents, factual accuracy, exam representations, or editorial choices in this book.

3. Trademark & Fair Use

Any references to university curriculum structures, course codes (e.g., CSE111), subject syllabi, or exam formats are made strictly under educational **fair use** principles to provide appropriate context and academic alignment for students studying these topics.

4. Accuracy & Corrections

While every effort has been made by student and educator contributors to verify factual accuracy, command syntax, and pedagogical clarity, the publishers provide this material on an “as-is” basis. Readers are encouraged to report any typographical or technical errors on the project's repository.

Contents

1	Computer Languages	1
1.1	Orientation: Communicating with Computers	1
1.2	Machine Language (Lowest Level)	2
1.2.1	Instruction Format & Structure	2
1.2.2	Advantages & Limitations of Machine Code	2
1.3	Assembly Language (Symbolic Low Level)	2
1.3.1	Symbolic Instructions and Syntax	2
1.3.2	One-to-One Relationship & The Assembler	3
1.4	High-Level Languages	3
1.4.1	Key Characteristics of High-Level Languages	3
1.5	Language Translators: Compiler, Interpreter, and Assembler	3
1.5.1	The Compiler	3
1.5.2	The Interpreter	4
1.5.3	The Assembler	4
1.6	The Program Development Cycle	4
1.7	The Compilation and Execution Process	5
1.7.1	The Complete Translation Pipeline	5
1.7.2	The CPU Instruction Cycle	5
1.8	Chapter Review & Key Takeaways	6
2	Computer Fundamentals	7
2.1	The Computer as a Programmable System	7
2.1.1	The Stored-Program Principle	7
2.1.2	Fundamental Operational Metrics	8
2.2	Historical Evolution and Computer Generations	8
2.2.1	Pioneering Mechanical Milestones	8
2.2.2	The Five Electronic Generations	9
2.2.3	Key Characteristics of Computers	9
2.3	Internal Organization of a Computer System	9
2.3.1	Functional Units Breakdown	10
2.3.2	System Buses: The Interconnect Highway	10
2.4	Hardware and Software Taxonomy	10
2.4.1	Software Classification	10
2.5	Classification of Computers	11
2.5.1	Classification by Processing Scale	11
2.5.2	Classification by Operational Principle	11
2.6	Chapter Review & Key Takeaways	12
3	Computer Hardware	13
3.1	The Physical Foundation of Computing	13
3.2	Processing Hardware: CPU and GPU Architecture	13
3.2.1	Central Processing Unit (CPU)	13
3.2.2	Graphics Processing Unit (GPU)	14

3.2.3	System Balance, Thermal Design, and Bottlenecks	15
3.3	Peripheral Hardware: Input and Output Devices	15
3.3.1	Input Peripherals	15
3.3.2	Output Peripherals	15
3.4	Memory and Storage Hardware	16
3.4.1	The Memory Hierarchy	16
3.4.2	Primary Memory: RAM and ROM	16
3.4.3	Secondary Storage: SSD vs. HDD	16
3.5	Chapter Review & Key Takeaways	17
4	Number Systems	18
4.1	Orientation: Positional Representation of Numbers	18
4.1.1	Base or Radix	18
4.1.2	The General Positional Evaluation Formula	18
4.2	The Four Key Number Systems in Computing	19
4.2.1	1. Binary (Base 2)	19
4.2.2	2. Decimal (Base 10)	19
4.2.3	3. Octal (Base 8)	20
4.2.4	4. Hexadecimal (Base 16)	20
4.3	Systematic Number Conversions	20
4.3.1	1. Converting from Any Base b to Decimal	20
4.3.2	2. Converting Decimal Integers to Any Base b (Repeated Division)	20
4.3.3	3. Converting Decimal Fractions to Any Base b (Repeated Multiplication)	21
4.3.4	4. Direct Grouping Conversions (Fast Method)	21
4.4	Chapter Review & Key Takeaways	22
5	Version Control with Git & GitHub	23
5.1	Orientation: The Principle of Version Control	23
5.1.1	The Distributed Architecture of Git	23
5.2	Git vs. GitHub: Local Tools vs. Cloud Collaboration	24
5.2.1	Relationship Between Local and Remote Repositories	24
5.3	Initial Setup and Configuration	24
5.3.1	Installing Git	24
5.3.2	Configuring User Identity	25
5.4	Local Repository Workflow: The Three-Tree Architecture	25
5.4.1	Step-by-Step Local Commands	25
5.4.2	Ignoring Unwanted Files: <code>.gitignore</code>	26
5.5	Branching and Collaborative Workflows	26
5.5.1	Branch Operations	26
5.6	Chapter Review & Key Takeaways	27
6	Modern AI Trends and Tools	28
6.1	Foundations of Artificial Intelligence	28
6.1.1	Historical Inception & The AI Hierarchy	28
6.2	Real-Life Applications of AI in Society	29
6.3	Generative Artificial Intelligence	29
6.3.1	Modalities of Generative AI	29
6.4	Modern AI Tools Suite	30
6.4.1	1. Text Generation & Conversational Assistants	30
6.4.2	2. Visual Content Generation Tools	30
6.4.3	3. Time-Based Media Generation	30
6.4.4	4. AI-Assisted Research Platforms	30

6.5	Prompt Engineering Architecture	30
6.6	Responsible and Ethical AI Practices	31
6.7	Chapter Review & Key Takeaways	32

Computer Languages

Unit 1 Overview & Learning Objectives

This chapter introduces the formal systems used to communicate instructions to digital computers. By the conclusion of this unit, students will be able to:

- Understand the structural hierarchy of computer languages from machine code to high-level abstractions.
- Differentiate between compilers, interpreters, and assemblers in terms of execution mechanics and error diagnostics.
- Trace the complete Program Development Cycle from problem specification to long-term software maintenance.
- Follow the step-by-step compilation and execution pipeline through preprocessing, compilation, assembly, linking, and loading.
- Analyze the internal CPU instruction cycle (Fetch, Decode, Execute, Store).

1.1 Orientation: Communicating with Computers

Computer languages are structured, formal systems used to define unambiguous instructions that a digital computer can store, translate, and execute. Computers cannot understand conversational human speech directly; at the deepest hardware level, digital processors operate strictly through binary electrical logic states (0 and 1).

Over the history of computer science, programming representations evolved from tedious, machine-dependent binary sequences to human-readable high-level languages like C, Java, and Python. However, regardless of how advanced or expressive a programming language is, **the CPU ultimately executes only machine-language binary instructions.**

Definition: Core Principles of Computer Languages

- **Binary Representation:** All computer instructions, memory addresses, and data values are internally represented as binary digits (**bits**), corresponding to physical electrical states (voltage thresholds).
- **Translation Requirement:** Because humans write in symbolic assembly or high-level code, translators (assemblers, compilers, or interpreters) are fundamentally required to produce executable machine code.
- **Machine Dependence:** Machine and assembly languages are tightly coupled to a specific processor family's Instruction Set Architecture (ISA), whereas high-level languages provide portability across diverse hardware architectures.
- **Syntax vs. Semantics:** A valid program must satisfy both **syntax** (grammatical

character structure and statement format) and **semantics** (logical meaning and expected operational behavior).

1.2 Machine Language (Lowest Level)

Machine language is the fundamental native programming language of any computer system. It consists entirely of strings of binary digits (0s and 1s) that are directly decoded and executed by the processor's electronic logic gates without requiring any software translator.

1.2.1 Instruction Format & Structure

A standard machine-language instruction is typically partitioned into two primary functional components:

$$\text{Machine Instruction} = [\text{Opcode}] + [\text{Operand(s)}] \quad (1.1)$$

- **Opcode (Operation Code):** A unique bit pattern that instructs the CPU's Control Unit which operation to perform (e.g., ADD, SUB, LOAD, STORE).
- **Operand:** Specifies the data or the physical memory addresses/registers on which the opcode must operate.

Worked Example: Binary Machine Code Representation

A typical 16-bit machine instruction might appear as:

10110000 01100001

Here, the high byte (10110000) could represent the opcode command "Move immediate byte into register AL", while the low byte (01100001) represents the immediate hexadecimal data value 0x61 (ASCII character 'a'). The exact interpretation depends entirely on the processor's Instruction Set Architecture (e.g., x86 vs. ARM).

1.2.2 Advantages & Limitations of Machine Code

- **Advantages:** Provides direct execution without runtime translation overhead; delivers maximum hardware efficiency and granular control over processor resources.
- **Limitations:** Extremely difficult for humans to write, memorize, and debug; highly prone to catastrophic errors (altering a single bit changes the entire instruction); completely non-portable between different CPU architectures.

1.3 Assembly Language (Symbolic Low Level)

To overcome the immense difficulty of writing raw binary strings, computer scientists created **Assembly Language**. Assembly is a symbolic, low-level language that replaces binary opcodes with intuitive, short English-like abbreviations called **mnemonics**.

1.3.1 Symbolic Instructions and Syntax

An assembly language statement directly models the underlying processor instruction format:

```
START:  MOV R1, #5           ; Load immediate constant 5 into register R1
```

```
ADD R2, R1, R1      ; Add R1 to R1 and place sum into register  
                     R2  
B    START          ; Unconditional branch back to label START
```

In this instruction sequence:

- MOV, ADD, and B are **mnemonics** representing opcodes.
- R1 and R2 are internal **CPU registers**.
- #5 represents an **immediate operand**.
- START: is a **symbolic label** representing a memory address.

1.3.2 One-to-One Relationship & The Assembler

Unlike high-level languages where a single statement can expand into dozens of machine operations, assembly language maintains an almost strict **one-to-one relationship**: one assembly statement translates into exactly one machine-code instruction.

Translating assembly language into machine-readable object code requires a dedicated translator known as an **Assembler**.

1.4 High-Level Languages

High-level programming languages (such as C, C++, Java, and Python) utilize natural English words, standard mathematical notation, and structured programming paradigms to hide processor-level hardware details from the software developer.

1.4.1 Key Characteristics of High-Level Languages

- **Abstraction**: Calculations are expressed naturally (e.g., `grossPay = hours * rate`) without requiring the programmer to manually allocate registers, manage memory addresses, or issue bus read commands.
- **Portability**: A high-level program written on an Intel x86 machine can be recompiled or run on an ARM-based computer without major code modifications.
- **Structured Constructs**: Features such as loops (`for`, `while`), conditional branching (`if-else`), modular functions, and object-oriented classes allow developers to organize complex software systems into manageable units.

1.5 Language Translators: Compiler, Interpreter, and Assembler

Computers cannot execute high-level statements directly. Therefore, specialized system programs called **language translators** convert human-authored source text into executable machine instructions.

1.5.1 The Compiler

A **compiler** is a translator that converts an entire high-level source program into machine code or intermediate object code **all at once prior to program execution**.

Definition: Core Phases of a Modern Compiler

1. **Lexical Analysis (Scanner)**: Reads source code characters and groups them into

atomic units called **tokens** (identifiers, literals, operators, keywords).

2. **Syntax Analysis (Parser):** Checks tokens against the formal grammatical rules of the language, constructing an Abstract Syntax Tree (AST). Rejects invalid grammar like `x = + 4;`.
3. **Semantic Analysis:** Validates logical meaning, scope, and type compatibility (e.g., ensuring a string is not assigned to an integer).
4. **Optimization:** Restructures intermediate code to run faster or consume less memory without changing program semantics.
5. **Code Generation:** Emits final target machine code or assembly instructions for the host architecture.

1.5.2 The Interpreter

An **interpreter** translates and executes a high-level program **line-by-line during runtime**. Instead of generating a standalone executable binary beforehand, the interpreter reads a statement, parses it, executes the requested operation immediately, and then proceeds to the next statement.

Comparison Criterion	Compiler (e.g., C, C++)	Interpreter (e.g., Python, JS)
Translation Timing	Entire program translated before execution	Translated statement-by-statement during runtime
Execution Speed	Fast native execution; no translation overhead	Slower execution due to runtime translation
Error Detection	Reports all compile-time syntax errors at once	Halts immediately when the first error is reached
Object Code	Generates permanent object/executable files	Does not generate standalone machine code files
Development Cycle	Requires edit-compile-link-run cycles	Interactive testing and rapid prototyping

1.5.3 The Assembler

An **assembler** translates symbolic assembly code into binary machine code. Most modern assemblers use a **Two-Pass Architecture**:

- **Pass 1 (Symbol Resolution):** Scans the entire file to find all label definitions and symbolic names, building a **Symbol Table** that maps each symbol to its assigned memory offset.
- **Pass 2 (Code Generation):** Scans the code a second time, replaces symbolic references with the actual memory addresses from the symbol table, and emits the binary object file.

1.6 The Program Development Cycle

Building reliable software requires following an organized engineering methodology known as the **Program Development Cycle**:

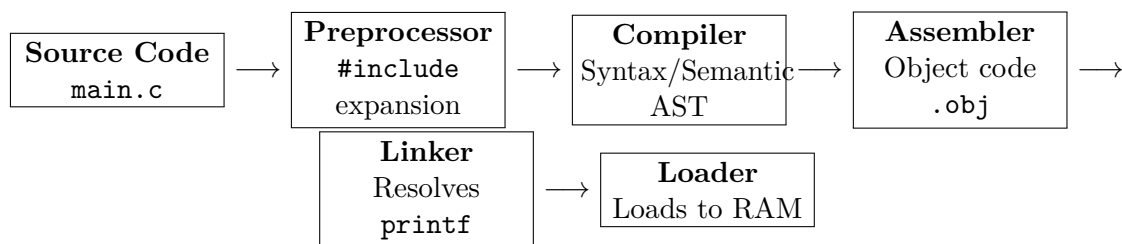
1. **Problem Definition:** Clearly identifying the problem, defining expected outputs, inputs, operational constraints, and target users.
2. **Problem Analysis:** Breaking down data requirements, validation checks (e.g., preventing negative pay hours), and boundary conditions.
3. **Algorithm Design:** Developing a step-by-step, finite logical solution represented via pseudocode or standard flowcharts.

4. **Coding:** Translating the algorithm into a concrete high-level programming language using proper variables, data types, and control structures.
5. **Translation:** Using a compiler or interpreter to identify syntax errors and produce runnable machine code.
6. **Testing & Debugging:** Running the software with diverse test data (normal, boundary, and erroneous data) to detect and correct syntax, runtime, and logic errors.
7. **Documentation:** Preparing user manuals and internal code comments explaining system architecture and maintenance guidelines.
8. **Maintenance:** Ongoing updates to patch bugs, adapt to new operating systems, and add new capabilities over time.

1.7 The Compilation and Execution Process

1.7.1 The Complete Translation Pipeline

When a programmer writes a program in a compiled language like C or C++, the source code undergoes several coordinated steps before executing on the processor:



- **Preprocessing:** Directives beginning with **#** are handled (macro substitution and inclusion of header files).
- **Compilation:** Converts preprocessed code into assembly code.
- **Assembly:** Converts assembly code into relocatable object code files containing machine instructions and symbol tables.
- **Linking:** Merges independent object files with pre-compiled standard system libraries (e.g., standard input/output routines) to produce a standalone executable binary.
- **Loading:** The operating system's loader allocates address space in RAM, copies the executable segments into memory, and passes control to the CPU.

1.7.2 The CPU Instruction Cycle

Once instructions reside in primary memory, the central processor repeatedly performs the foundational **Instruction Cycle**:

1. **Fetch:** The Control Unit retrieves the instruction from the memory address specified by the **Program Counter (PC)** register into the **Instruction Register (IR)**.
2. **Decode:** The decoder circuit interprets the opcode bits to determine the required operation and operand locations.
3. **Execute:** The Arithmetic Logic Unit (ALU) or control circuits execute the requested operation (such as adding two register values).
4. **Store (Write-back):** The result of the computation is written back to a destination CPU register or memory location.

1.8 Chapter Review & Key Takeaways

Summary Checklist

- Machine language consists exclusively of binary digits (0 and 1) and runs directly on the CPU without translation.
- Assembly language replaces binary opcodes with mnemonics (MOV, ADD) and uses an assembler to generate object code.
- Compilers translate the whole program at once before execution; interpreters evaluate code statement-by-statement at runtime.
- The compilation pipeline proceeds from Preprocessor → Compiler → Assembler → Linker → Loader.
- The CPU instruction execution cycle consists of four continuous phases: Fetch → Decode → Execute → Store.

Computer Fundamentals

Unit 2 Overview & Learning Objectives

This chapter covers the architectural origins, evolution, and core structural organization of modern electronic computing systems. By the end of this unit, students will be able to:

- Understand the stored-program architecture proposed by John von Neumann in 1945.
- Trace the five technological generations of computing from vacuum tubes to modern artificial intelligence.
- Analyze the five foundational functional units of a computer system and system interconnect buses.
- Distinguish between system software, application software, utility programs, and low-level firmware.
- Classify computer systems by processing scale, operational physical principles, and domain purpose.

2.1 The Computer as a Programmable System

A **computer** is an electronic, programmable machine designed to accept raw data as input, process it systematically according to a stored sequence of instructions, produce meaningful information as output, and retain results in memory or secondary storage for future use.

2.1.1 The Stored-Program Principle

Virtually all modern general-purpose digital computers are based on the **von Neumann architecture**, formalized by mathematician John von Neumann in 1945. The revolutionary breakthrough of this design was the **stored-program concept**:

Both program instructions and the data upon which they operate are held together within the same unified primary memory space and fetched sequentially by the CPU.

Before this architecture, early calculating machines were hardwired to perform only a single fixed task; altering the machine's function required manual rewiring of physical circuits and plugboards.

Definition: Data vs. Information

- **Data:** Unprocessed, raw facts, numbers, or observations that carry no contextual meaning on their own (e.g., 72, "LPU", 101100).
- **Information:** Data that has been processed, structured, organized, or contextualized into a meaningful form that enables decision-making (e.g., "The average exam score is 72%").

2.1.2 Fundamental Operational Metrics

- **Storage Capacity:** Measured in bytes (1 Byte = 8 bits). Standard decimal multiples are Kilobytes (10^3), Megabytes (10^6), Gigabytes (10^9), and Terabytes (10^{12}). Binary computing standards use Kibibytes ($2^{10} = 1,024$ B), Mebibytes (2^{20} B), and Gibibytes (2^{30} B).
- **Processing Frequency:** Clock frequency is measured in Hertz (Hz), where 1 Hertz = 1 cycle per second. Modern microprocessors operate in Gigahertz (1 GHz = 10^9 cycles per second).
- **Data Transfer Rates:** The transmission speed across communication channels, measured in bits per second (bps) or bytes per second (Bps).

2.2 Historical Evolution and Computer Generations

The progression of calculating machinery spans centuries, transitioning from manual aids to mechanical gear systems and finally to solid-state electronic integrated circuits.

2.2.1 Pioneering Mechanical Milestones

- **The Abacus:** An ancient manual calculating frame using beads on rods to perform basic arithmetic operations.
- **The Pascaline (1642):** Invented by French mathematician Blaise Pascal, the Pascaline was the first mechanical calculating device based on rotating geared wheels to perform addition and subtraction.
- **The Analytical Engine (1830s):** Designed by Charles Babbage (celebrated as the “Father of the Computer”), this mechanical general-purpose design anticipated the fundamental architecture of modern computers, incorporating an arithmetic processing mill, a memory store, and punched card input/output.
- **Herman Hollerith’s Tabulating Machine (1890):** Automated data processing using punched cards for the 1890 United States Census, cutting processing time from eight years down to under one year and leading to the establishment of IBM.

2.2.2 The Five Electronic Generations

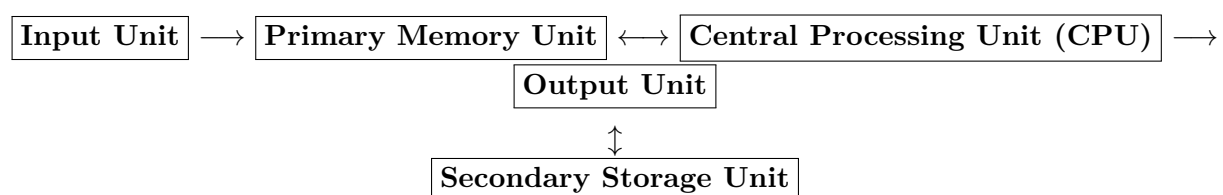
Gen	Era	Core Electronic Tech	Characteristics, Architecture & Software
1st	1940–1956	Vacuum Tubes	ENIAC, UNIVAC. Consumed huge electricity, produced immense heat, suffered constant tube burnouts. Programmed in binary machine language.
2nd	1956–1963	Transistors	Replaced vacuum tubes with solid-state semiconductor transistors. Far smaller, faster, cooler, and more reliable. Introduced assembly language and early high-level languages (FORTRAN, COBOL).
3rd	1964–1971	Integrated Circuits (ICs)	Multiple transistors fabricated on a single silicon chip (Jack Kilby & Robert Noyce). Introduced monitors, keyboards, and early time-sharing operating systems.
4th	1971–Present	Microprocessors (VLSI)	The entire CPU fabricated onto a single compact integrated circuit (Intel 4004 in 1971). Led to personal computers (PCs), laptops, mobile devices, and the Internet.
5th	Emerging	Artificial Intelligence	Ongoing focus on massive parallel processing, neural networks, natural language comprehension, and autonomous machine learning.

2.2.3 Key Characteristics of Computers

- **Speed:** Capable of executing billions of mathematical and logical operations per second (10^9 operations/sec).
- **Accuracy & GIGO:** High precision guaranteed by hardware logic. The rule **Garbage In, Garbage Out (GIGO)** emphasizes that faulty input data or erroneous software logic inevitably produces incorrect results.
- **Diligence & Consistency:** Operates continuously without mental fatigue, boredom, or loss of computational concentration.
- **Versatility:** A single machine can execute entirely different tasks (e.g., word processing, video playback, mathematical modeling) simply by loading different programs into memory.
- **Fundamental Limitations:** Computers lack independent common sense, human empathy, original intuition, and moral judgment; their operations are strictly bound by the algorithms and data provided by human developers.

2.3 Internal Organization of a Computer System

At the architectural level, every digital computer system is organized into five coordinated functional units interconnected by system buses:



2.3.1 Functional Units Breakdown

1. **Input Unit:** Receives data and commands from external users or physical sensors and translates them into internal binary signals (e.g., keyboard, mouse, scanner).
2. **Central Processing Unit (CPU):** The brain of the computer system that executes instructions:
 - **ALU (Arithmetic Logic Unit):** Executes arithmetic (+, −, ×, ÷) and Boolean logical comparisons (AND, OR, NOT, <, >, =).
 - **Control Unit (CU):** The manager that coordinates hardware operations, sequencing instruction execution, and issuing read/write timing signals.
 - **Registers:** High-speed internal memory cells holding active instructions, operands, and memory pointers during execution.
3. **Memory Unit (Primary Storage):** Holds currently executing programs and their working variables (RAM, ROM, Cache).
4. **Secondary Storage Unit:** Retains operating system files, application programs, and user documents permanently when the system is powered down (SSD, HDD, Flash).
5. **Output Unit:** Translates processed binary data into human-understandable visual, printed, or auditory forms (e.g., monitors, printers, speakers).

2.3.2 System Buses: The Interconnect Highway

A **bus** is a set of parallel physical electrical pathways that carry binary signals between the CPU, primary memory, and input/output controllers.

- **Data Bus (Bidirectional):** Carries the actual data words and machine instructions. Its width (e.g., 32-bit or 64-bit) determines how many bits can be transferred in a single clock cycle.
- **Address Bus (Unidirectional):** Transmits the memory location or peripheral I/O port address that the CPU wishes to access. An n -bit address bus can directly address up to 2^n unique memory locations.
- **Control Bus:** Carries synchronization signals and operational commands, including **Memory Read**, **Memory Write**, **I/O Read**, **I/O Write**, and **Interrupt Request**.

2.4 Hardware and Software Taxonomy

A complete computing system represents a tight partnership between physical components (**hardware**) and programmed instructions (**software**).

2.4.1 Software Classification

- **System Software:** Manages the underlying hardware resources and provides a standard execution platform for applications:
 - *Operating Systems:* Manages processes, CPU scheduling, memory allocation, storage filesystems, and device security (e.g., Windows, Linux, macOS).
 - *Device Drivers:* Specialized translation programs allowing the OS to communicate with specific hardware models (e.g., graphics drivers, printer drivers).
 - *Utility Software:* Performs system maintenance, disk defragmentation, virus scanning, and data compression.
- **Application Software:** End-user software designed to accomplish specific real-world tasks

(e.g., Microsoft Word, Google Chrome, Adobe Photoshop).

- **Firmware:** Semi-permanent software burned into non-volatile ROM chips that provides low-level hardware control during power-on initialization (e.g., BIOS, UEFI).

2.5 Classification of Computers

Modern computers are classified across three dimensions: computational scale, operational principle, and purpose.

2.5.1 Classification by Processing Scale

- **Supercomputers:** The most powerful systems on Earth, utilizing thousands of parallel processors to perform trillions of floating-point operations per second (PFLOPS) for nuclear physics, global climate modeling, and molecular simulation.
- **Mainframe Computers:** Massive, ultra-reliable enterprise systems optimized for high input/output throughput and continuous processing of millions of simultaneous financial transactions (used by major banks, airlines, and social security).
- **Servers:** High-availability computers that provide shared services, web hosting, email, and database storage to client machines across a local network or the Internet.
- **Workstations:** High-performance single-user desktop systems equipped with professional GPUs, multi-core CPUs, and large RAM for computer-aided design (CAD), video rendering, and simulation.
- **Personal Computers (PCs) & Laptops:** General-purpose computers designed for everyday productivity, web browsing, study, and entertainment by individual users.
- **Embedded Computers:** Compact microcontrollers integrated directly into larger physical products (automobiles, smart appliances, medical devices) to perform dedicated control and monitoring tasks, frequently operating under real-time constraints.

2.5.2 Classification by Operational Principle

- **Analog Computers:** Measure continuous physical quantities (e.g., electrical voltage, fluid pressure, rotational velocity) directly without converting to digital bits.
- **Digital Computers:** Process discrete digital states (0 and 1). Highly precise, programmable, and universally used in contemporary computing.
- **Hybrid Computers:** Combine analog measurement circuits with digital processing and storage (e.g., intensive-care patient monitoring equipment).

2.6 Chapter Review & Key Takeaways

Summary Checklist

- The von Neumann architecture stores both program instructions and data in unified primary memory.
- Computer generations: 1st (Vacuum tubes) → 2nd (Transistors) → 3rd (ICs) → 4th (Microprocessors/Intel 4004) → 5th (Artificial Intelligence).
- The CPU consists of the Arithmetic Logic Unit (ALU), the Control Unit (CU), and high-speed registers.
- System buses include the bidirectional Data Bus, the unidirectional Address Bus, and the Control Bus.
- Software divides into System Software (OS, drivers), Application Software, Utilities, and low-level Firmware.

Computer Hardware

Unit 3 Overview & Learning Objectives

This chapter explores the physical, electronic, and electromechanical components that form a digital computer. By the end of this unit, students will be able to:

- Analyze the distinct architectural designs of the Central Processing Unit (CPU) and Graphics Processing Unit (GPU).
- Understand the operational roles of internal processor registers, multi-level caches, and system clocks.
- Evaluate the full spectrum of input and output peripherals, including accessibility devices and signal converters.
- Master the Computer Memory Hierarchy across registers, cache, DRAM, SRAM, ROM, SSDs, HDDs, and archival tape.
- Compute precise binary memory capacities and explain the mechanics of volatile vs. non-volatile storage.

3.1 The Physical Foundation of Computing

Computer hardware encompasses the physical electronic circuitry, microchips, buses, and mechanical devices that execute instructions. Hardware operates under the foundational **Input–Process–Output–Storage (IPO-S)** model:

- **Input:** Translates real-world human interactions or physical signals into digital binary data (0s and 1s).
- **Process:** Transforms and manipulates binary data through arithmetic calculations, logical comparisons, and control flows directed by the processor.
- **Output:** Converts processed binary information into sensory forms usable by humans (screens, speakers, paper printouts) or control actions directed to machinery (actuators).
- **Storage:** Retains instructions and operational data across immediate primary access (RAM, cache) and long-term non-volatile secondary persistence (SSDs, hard drives).

3.2 Processing Hardware: CPU and GPU Architecture

Modern computer systems rely on specialized processor architectures tailored to different computational workloads.

3.2.1 Central Processing Unit (CPU)

The **CPU** is the general-purpose computational engine responsible for managing system resources, executing the operating system kernel, and coordinating application programs.

Definition: Key Architectural Sub-components of the CPU

- **Control Unit (CU):** The conductor of the CPU. It fetches instructions from memory, decodes them, and coordinates the flow of data by generating timed electrical control signals across buses.
- **Arithmetic Logic Unit (ALU):** Performs all mathematical arithmetic (addition, subtraction, multiplication) and Boolean logic evaluations (AND, OR, NOT, and comparison tests like $<$, $>$, $=$).
- **Processor Registers:** Ultra-high-speed memory cells fabricated directly inside the CPU core. Crucial registers include:
 - **Program Counter (PC):** Holds the memory address of the **next** instruction to be fetched and executed.
 - **Instruction Register (IR):** Holds the instruction **currently being decoded and executed**.
 - **Accumulator / General-Purpose Registers:** Temporarily store arithmetic operands and intermediate computational results.
- **Cache Memory:** Small, extremely fast static RAM placed directly on the CPU die to store frequently accessed data and instructions:
 - **Level 1 (L1) Cache:** The smallest, fastest, and closest cache dedicated to individual cores (typically 32–64 KB).
 - **Level 2 (L2) Cache:** Larger than L1, slightly slower, dedicated or shared between core pairs.
 - **Level 3 (L3) Cache:** The largest on-chip cache (megabytes), shared across all processor cores.

3.2.2 Graphics Processing Unit (GPU)

While a modern CPU features between 4 and 64 large, highly versatile cores designed for low-latency sequential decision-making, a **GPU** features **thousands of smaller, simpler execution cores** designed to execute the same instruction across vast streams of data simultaneously.

Comparison Factor	CPU (Central Processing Unit)	GPU (Graphics Processing Unit)
Core Architecture	Few powerful cores with large caches	Thousands of small, lightweight cores
Optimization Goal	Low latency for sequential branching	High throughput for massively parallel math
Ideal Workloads	Operating systems, databases, sequential code	3D graphics rendering, video encoding, Deep Learning
Dedicated Memory	Uses system main memory (DRAM)	Uses dedicated high-bandwidth VRAM

Worked Example: Why GPUs Dominate Deep Learning Training

Training an artificial neural network involves multiplying and adjusting millions of interconnected matrix weights. Because matrix multiplications consist of independent, repetitive arithmetic calculations that can happen simultaneously, a GPU's thousands of parallel

cores can process entire layers at once, achieving speedups of $10\times$ to $100\times$ over traditional CPUs.

3.2.3 System Balance, Thermal Design, and Bottlenecks

A computer's overall speed is not dictated by CPU clock speed alone. A high-speed CPU paired with inadequate RAM or a slow mechanical hard drive will experience **bottlenecks**, spending clock cycles idle while waiting for data. Furthermore, modern microprocessors generate significant heat and require active cooling (heat sinks, thermal paste, cooling fans, or liquid loops) to prevent **thermal throttling** (automatic downclocking to prevent physical damage).

3.3 Peripheral Hardware: Input and Output Devices

Peripherals connect the computer's central processor to the outside world.

3.3.1 Input Peripherals

- **Keyboards & Pointing Devices:** Keyboards convert keystrokes into unique binary scan codes. Mice, trackballs, and touchpads detect physical motion or capacitance to manipulate an on-screen coordinate pointer.
- **Touchscreen Displays:** Highly versatile hybrid devices that function simultaneously as an **input device** (detecting touch gestures) and an **output device** (displaying pixels).
- **Scanners and Optical Character Recognition (OCR):** Scanners digitize printed physical documents into bitmap images; OCR software parses image patterns into editable text.
- **Microphones & Analog-to-Digital Converters (ADC):** Microphones detect physical sound vibrations; an ADC continuously samples the analog electrical waveform into digital binary audio samples.
- **Biometric Devices:** Capture biological characteristics (fingerprints, facial geometry, iris patterns) for identity verification and access security.
- **Sensors:** Measure continuous environmental variables (temperature, humidity, acceleration, proximity) for monitoring and automated control systems.

3.3.2 Output Peripherals

- **Monitors & Displays:** Render text, images, and video as a grid of picture elements (**pixels**). Key specifications include **Resolution** (total pixel dimensions, e.g., 1920×1080 Full HD, 3840×2160 4K) and **Refresh Rate** (frequency of screen updates, measured in Hertz, e.g., 60 Hz, 144 Hz).
- **Printers:**
 - *Laser Printers:* Utilize powdered plastic toner, an electrostatic drum, and a high-heat fuser to print crisp text documents at high speed.
 - *Inkjet Printers:* Spray microscopic droplets of liquid ink through precision nozzles; ideal for color photographs and graphic art.
- **Audio Output & Digital-to-Analog Converters (DAC):** A DAC transforms digital binary audio streams into continuous electrical voltages to drive speakers or headphones.
- **Actuators:** Electromechanical output transducers that convert digital control signals into physical mechanical motion (e.g., spinning an industrial motor, adjusting a robotic arm, opening a hydraulic valve).
- **Haptic Devices:** Provide physical tactile feedback to the user through precision vibration

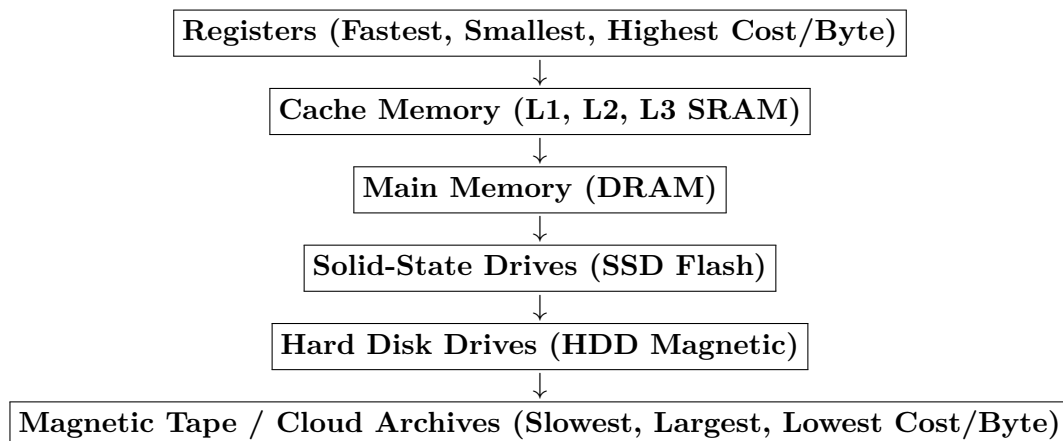
motors (e.g., smartphone notifications, game controller rumble).

3.4 Memory and Storage Hardware

Memory holds the binary instructions and data needed before, during, and after CPU processing.

3.4.1 The Memory Hierarchy

Because no single memory technology is simultaneously ultra-fast, massive, and cheap, computer architectures employ a multi-level **Memory Hierarchy**:



3.4.2 Primary Memory: RAM and ROM

- **Random-Access Memory (RAM):** The primary working memory of the computer. RAM is strictly **volatile**, meaning its contents vanish completely when electrical power is disconnected.
 - **Dynamic RAM (DRAM):** Stores each bit as an electrical charge in a microscopic capacitor and transistor cell. Because capacitors naturally leak charge over time, DRAM requires **periodic electrical refreshing** hundreds of times per second. It provides high storage density at moderate cost and serves as main system memory.
 - **Static RAM (SRAM):** Stores each bit using a flip-flop circuit consisting of 4 to 6 transistors. It requires no refreshing as long as power is maintained, making it substantially faster than DRAM. Because it is physically larger and more expensive per bit, SRAM is reserved for high-speed CPU caches.
- **Read-Only Memory (ROM):** **Non-volatile** memory whose contents are permanently retained even when the computer is turned off. It houses essential low-level startup firmware (BIOS/UEFI) that boots the system hardware.
- **Flash Memory:** Non-volatile solid-state memory that can be erased and reprogrammed in electrical blocks. Widely used in SSDs, USB thumb drives, and smartphone storage.

3.4.3 Secondary Storage: SSD vs. HDD

- **Hard Disk Drive (HDD):** Stores binary data magnetically on rapidly spinning circular platters (e.g., 5400 or 7200 RPM) accessed by mechanical floating read/write actuator arms. HDDs provide economical high capacity, but have mechanical access latency (seek time and rotational delay) and can be permanently damaged by physical drops or shocks.
- **Solid-State Drive (SSD):** Stores data in non-volatile flash memory cells with **no moving mechanical parts**. Delivers near-instantaneous access speeds, silent operation, low power consumption, and high resistance to physical vibration. Modern SSD controllers use sophisticated **wear-leveling algorithms** to distribute write cycles evenly across memory blocks,

maximizing drive lifespan.

- **Magnetic Tape:** Sequential magnetic media primarily utilized for high-capacity, low-cost long-term regulatory and disaster-recovery archival backups.

3.5 Chapter Review & Key Takeaways

Summary Checklist

- The CPU handles sequential logic and OS control; the GPU uses thousands of cores for massive parallel matrix computations.
- The Program Counter (PC) stores the next instruction address; the Instruction Register (IR) holds the current instruction.
- DRAM requires periodic electrical refreshing and serves as main RAM; SRAM is faster, requires no refresh, and forms CPU cache.
- ROM is non-volatile and holds startup firmware; RAM is volatile and loses contents when powered down.
- SSDs utilize flash memory with no moving parts, outperforming mechanical HDDs in speed and physical durability.

Number Systems

Unit 4 Overview & Learning Objectives

This chapter establishes the mathematical principles of positional numeral systems used in computing. By the end of this unit, students will be able to:

- Understand the positional value formula and the role of the radix point, MSD, and LSD.
- Master binary, octal, decimal, and hexadecimal representations.
- Convert between any arbitrary base and the decimal number system.
- Execute rapid direct conversions between binary, octal, and hexadecimal using 3-bit and 4-bit grouping rules.
- Perform integer conversions via repeated division and fractional conversions via repeated multiplication.

4.1 Orientation: Positional Representation of Numbers

A **number system** is a formal method of representing quantities using an agreed-upon set of symbols called **digits**. Modern computing is founded entirely on **positional number systems**, where the value contributed by any digit depends both on the symbol itself and its specific position within the number string.

4.1.1 Base or Radix

The **base** (or **radix**), denoted by b , defines the total number of distinct allowable digit symbols in that system. A base- b system uses digits strictly ranging from 0 up to $b - 1$.

- **Binary (Base 2):** Uses two digits: 0, 1.
- **Octal (Base 8):** Uses eight digits: 0, 1, 2, 3, 4, 5, 6, 7.
- **Decimal (Base 10):** Uses ten digits: 0, 1, 2, 3, 4, 5, 6, 7, 8, 9.
- **Hexadecimal (Base 16):** Uses sixteen symbols: 0–9 and A–F.

4.1.2 The General Positional Evaluation Formula

Any real number represented in a base- b positional system can be evaluated into its equivalent decimal magnitude using the sum of products of each digit multiplied by its integer positional power of the base:

$$(d_n d_{n-1} \dots d_1 d_0 . d_{-1} d_{-2} \dots)_b = \sum_{i=-\infty}^n d_i \times b^i \quad (4.1)$$

- For integers, positional exponents increase positively from right to left starting at index $i = 0$ (the units position, with weight $b^0 = 1$).
- After the **radix point**, fractional exponents decrease negatively from left to right ($b^{-1}, b^{-2}, b^{-3}, \dots$).

- **MSD (Most Significant Digit):** The leftmost non-zero digit carrying the highest positional weight.
- **LSD (Least Significant Digit):** The rightmost digit carrying the lowest positional weight.

4.2 The Four Key Number Systems in Computing

4.2.1 1. Binary (Base 2)

Binary is the fundamental language of digital electronic computers because two-state electrical switches (transistors operating in saturation or cutoff) are physically simple, highly reliable, and resistant to electrical noise.

- **Symbols:** 0, 1.
- **Integer Weights:** $2^0 = 1$, $2^1 = 2$, $2^2 = 4$, $2^3 = 8$, $2^4 = 16$, $2^5 = 32$, $2^6 = 64$, $2^7 = 128$.
- **Fractional Weights:** $2^{-1} = 0.5$, $2^{-2} = 0.25$, $2^{-3} = 0.125$, $2^{-4} = 0.0625$.

Worked Example: Evaluating a Binary Positional Number

Convert $(1011.01)_2$ to its decimal equivalent:

$$\begin{aligned}(1011.01)_2 &= (1 \times 2^3) + (0 \times 2^2) + (1 \times 2^1) + (1 \times 2^0) + (0 \times 2^{-1}) + (1 \times 2^{-2}) \\ &= 8 + 0 + 2 + 1 + 0 + 0.25 \\ &= \mathbf{11.25}_{10}\end{aligned}$$

Definition: Binary Arithmetic Addition Rules

$$\begin{aligned}0 + 0 &= 0 \\ 0 + 1 &= 1 \\ 1 + 0 &= 1 \\ 1 + 1 &= 10_2 \quad (\text{Sum 0 with a carry of 1 to next column}) \\ 1 + 1 + 1 &= 11_2 \quad (\text{Sum 1 with a carry of 1 to next column})\end{aligned}$$

4.2.2 2. Decimal (Base 10)

Decimal is the universal human counting system. In computing, decimal is used at user interfaces for inputs and displayed reports. However, internally, computers do not store numbers in decimal.

Common Pitfall: Why Floating-Point Decimals Experience Rounding Errors

In decimal, fractions like $1/10 = 0.1_{10}$ terminate cleanly. But in binary, because 10 is not a pure power of 2, the fraction 0.1_{10} becomes an **infinitely repeating binary fraction**:

$$0.1_{10} = (0.0001100110011 \dots)_2$$

When stored in finite registers, the number must be truncated, resulting in tiny mathematical rounding discrepancies across repeated calculations.

4.2.3 3. Octal (Base 8)

Octal provides a convenient, human-friendly shorthand representation for binary strings. Because $8 = 2^3$, **exactly three binary bits correspond to one octal digit**.

- **Digits Allowed:** 0, 1, 2, 3, 4, 5, 6, 7 (the digits 8 and 9 are strictly illegal).
- **Weights:** $8^0 = 1$, $8^1 = 8$, $8^2 = 64$, $8^3 = 512$.
- **Practical Application:** Traditional Unix/Linux file access permissions are expressed in octal (e.g., permission 755, where octal digit $7_8 = 111_2$ represents read, write, and execute permissions).

4.2.4 4. Hexadecimal (Base 16)

Hexadecimal is the primary compact notation used across modern software engineering, computer architecture, and networking. Because $16 = 2^4$, **exactly four binary bits map to one hexadecimal digit**.

- **Digits Allowed:** 0, 1, 2, 3, 4, 5, 6, 7, 8, 9 and the letters A = 10, B = 11, C = 12, D = 13, E = 14, F = 15.
- **Weights:** $16^0 = 1$, $16^1 = 16$, $16^2 = 256$, $16^3 = 4096$.
- **Syntax Conventions:** Often prefixed with 0x in C/C++/Python (e.g., 0x2AF) or suffixed with h in assembly (e.g., 2AFh).
- **Applications:** Displaying memory addresses (e.g., 0x7FFF), inspecting compiled machine code, defining MAC hardware addresses, and representing RGB colors in web design (e.g., #FF0000 represents pure red).

4.3 Systematic Number Conversions

4.3.1 1. Converting from Any Base b to Decimal

Multiply each digit by its positional weight b^i and compute the arithmetic sum:

Worked Example: Hexadecimal to Decimal

Convert $(2AF)_{16}$ to decimal:

$$\begin{aligned}(2AF)_{16} &= (2 \times 16^2) + (10 \times 16^1) + (15 \times 16^0) \\ &= (2 \times 256) + (10 \times 16) + (15 \times 1) \\ &= 512 + 160 + 15 = \mathbf{687}_{10}\end{aligned}$$

4.3.2 2. Converting Decimal Integers to Any Base b (Repeated Division)

Divide the decimal integer repeatedly by the target base b , recording the integer quotient and the remainder at each step. Terminate when the quotient becomes 0. The final answer is obtained by reading the remainders **from bottom (last remainder = MSD) to top (first remainder = LSD)**.

Worked Example: Decimal 156_{10} to Binary

$$\begin{array}{rcl}
156 \div 2 = 78 & \text{remainder } 0 & \text{(LSD)} \\
78 \div 2 = 39 & \text{remainder } 0 & \\
39 \div 2 = 19 & \text{remainder } 1 & \\
19 \div 2 = 9 & \text{remainder } 1 & \\
9 \div 2 = 4 & \text{remainder } 1 & \\
4 \div 2 = 2 & \text{remainder } 0 & \\
2 \div 2 = 1 & \text{remainder } 0 & \\
1 \div 2 = 0 & \text{remainder } 1 & \text{(MSD)} \implies (10011100)_2
\end{array}$$

4.3.3 3. Converting Decimal Fractions to Any Base b (Repeated Multiplication)

Multiply the fractional portion repeatedly by the target base b . Record the resulting integer part at each step and continue multiplying only the remaining fractional component. Read the integer parts **from top (first generated = MSD) to bottom (last generated = LSD)**.

4.3.4 4. Direct Grouping Conversions (Fast Method)

Because $8 = 2^3$ and $16 = 2^4$, conversions between binary, octal, and hexadecimal bypass decimal arithmetic entirely:

- **Binary $\rightarrow$ Octal:** Partition bits into groups of **three** starting from the radix point (adding leading or trailing zeros if needed):

$$\underbrace{010}_2 \underbrace{011}_3 \underbrace{100}_4 {}_2 \implies (234)_8$$

- **Binary $\rightarrow$ Hexadecimal:** Partition bits into groups of **four** starting from the radix point:

$$\underbrace{1001}_9 \underbrace{1100}_{12=C} {}_2 \implies (9C)_{16}$$

- **Octal $\leftrightarrow$ Hexadecimal:** Convert each octal digit to 3 binary bits, then regroup the binary sequence into 4-bit blocks to yield the hexadecimal equivalent.

Worked Example: Integrated Equivalency Verification

Notice that:

$$156_{10} = (10011100)_2 = (234)_8 = (9C)_{16}$$

All four representations denote the exact same mathematical quantity.

4.4 Chapter Review & Key Takeaways

Summary Checklist

- A positional number system evaluates value through $\sum(d_i \times b^i)$.
- Binary is base 2 (0, 1); Octal is base 8 (0–7); Decimal is base 10 (0–9); Hexadecimal is base 16 (0–9, A–F).
- Octal digits map directly to 3 binary bits; Hexadecimal digits map directly to 4 binary bits.
- Decimal integers convert via repeated division (read remainders bottom-to-top); fractions convert via repeated multiplication (read integers top-to-bottom).
- An unsigned 8-bit byte can represent decimal values from 0 up to 255 ($11111111_2 = FF_{16}$).

Version Control with Git & GitHub

Unit 5 Overview & Learning Objectives

This chapter introduces professional software version tracking and collaborative team development using Git and GitHub. By the end of this unit, students will be able to:

- Understand the distributed version control model and its advantages over centralized systems.
- Configure local Git author identity and understand configuration scopes.
- Navigate the Git three-tree architecture: Working Directory, Staging Area, and Local Repository.
- Master daily terminal operations: `init`, `status`, `add`, `commit`, `log`, and `restore`.
- Utilize branches for isolated feature development and synchronize local repositories with remote GitHub remotes.

5.1 Orientation: The Principle of Version Control

Version Control is a systematic software engineering methodology for recording modifications made to files over time, enabling developers to review historical revisions, compare changes, collaborate concurrently without overwriting code, and restore previous operational states whenever bugs arise.

5.1.1 The Distributed Architecture of Git

Created by ****Linus Torvalds in 2005**** to manage development of the Linux kernel, **Git** is a free, open-source **Distributed Version Control System (DVCS)**.

Unlike legacy centralized version control systems (such as SVN or CVS) where developers must connect to a single central server to commit changes or view project history, **every Git clone on a developer's machine is a complete repository** containing the entire project history, branches, and commit metadata. Developers can inspect logs, create branches, and commit snapshots completely offline.

Definition: Key Version Control Terminology

- **Repository (Repo):** A directory containing project files along with a hidden administrative folder (`.git`) that tracks the entire historical record of changes.
- **Commit:** A saved permanent snapshot of staged modifications, uniquely identified by a 40-character SHA-1 hash.
- **Branch:** A lightweight, movable pointer to a specific commit representing an independent line of development.

- **Remote Repository:** A copy of the repository hosted on a cloud server or local network (e.g., on GitHub) used for backup and multi-developer collaboration.

5.2 Git vs. GitHub: Local Tools vs. Cloud Collaboration

A common beginner misconception is confusing Git with GitHub. They are distinct technologies:

- **Git:** The command-line software tool installed locally on your operating system that manages snapshots, staging, branches, and version history.
- **GitHub:** A cloud-based web hosting platform that stores remote Git repositories, offering collaboration features such as web-based code viewing, Pull Requests, automated testing (CI/CD), and team permission management.

Aspect	Git	GitHub
Type	Command-line version control software	Cloud-hosted web collaboration service
Installation	Installed locally on personal computers	Accessed via web browser and APIs
Network Need	100% functional offline for local work	Requires internet to sync with cloud
Maintainer	Open-source community (initiated by Linux)	Commercial enterprise (subsidiary of Microsoft)

5.2.1 Relationship Between Local and Remote Repositories

Local and remote repositories remain separate copies whose histories are explicitly synchronized through Git networking commands:

- **git push:** Uploads local branch commits to the remote repository.
- **git fetch:** Downloads new remote commits into local tracking references without altering working files.
- **git pull:** Fetches remote updates and immediately merges them into the current active branch.
- **git clone <URL>:** Downloads a full remote repository and its entire history into a brand new local folder.

Common Pitfall: The Synchronization Golden Rule

Saving a file inside an editor (e.g., **Ctrl+S**) only updates the local file on disk. Creating a commit (**git commit**) records the snapshot locally on your machine. Nothing is uploaded to GitHub until you explicitly run **git push**.

5.3 Initial Setup and Configuration

5.3.1 Installing Git

- **Windows:** Download from the official website (<https://git-scm.com/>); installs Git along with the **Git Bash** terminal emulator.
- **macOS:** Installed via Apple Command Line Developer Tools:

```
xcode-select --install
```

- **Linux (Debian/Ubuntu):** Installed via package manager:

```
sudo apt update && sudo apt install git
```

- **Verification:** Check version in the terminal:

```
git --version
```

5.3.2 Configuring User Identity

Before creating commits, developers must configure their author identity. Git records this information permanently inside every commit:

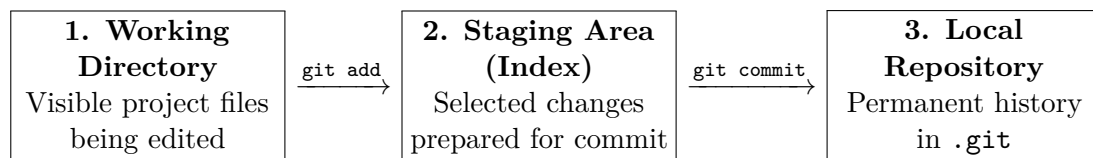
```
git config --global user.name "Your Name"
git config --global user.email "your.email@example.com"
```

The `-global` flag writes settings to the user's home configuration file (`~/.gitconfig`), applying them to all repositories on that machine. To verify active settings:

```
git config --list --show-origin
```

5.4 Local Repository Workflow: The Three-Tree Architecture

Git organizes files across three logical areas:



5.4.1 Step-by-Step Local Commands

1. **Initializing a Repository:** Navigate into your project directory and initialize Git tracking:

```
mkdir my-project && cd my-project
git init -b main
```

This creates the hidden `.git` directory, which contains object stores, branches, HEAD references, and configuration.

2. **Checking Repository Status:**

```
git status
```

Reports which branch is active, which files are staged for commit, which files are modified but unstaged, and which files are untracked.

3. **Staging Files:**

```
git add README.md          # Stages a specific file
git add .                  # Stages all modified and new files in
                           # directory
```

If a staged file is edited again before committing, the newly added modifications remain in the working tree until `git add` is executed again.

4. Unstaging Modifications:

```
git restore --staged README.md
```

Removes the file from the staging index while leaving all edits safe in the working directory.

5. Creating a Commit:

```
git commit -m "Add project overview to README"
```

Creates a new immutable snapshot. A good commit message uses the imperative mood (e.g., "Fix login validation bug"), accurately describing the functional change.

6. Viewing Project History:

```
git log --oneline
```

Displays past commits from newest to oldest in a concise single-line format showing the truncated SHA-1 hash and commit message.

5.4.2 Ignoring Unwanted Files: .gitignore

Certain files should never be committed into version control (such as temporary editor files, log files, compiled binaries, and private API keys or database passwords). Creating a plain-text file named **.gitignore** in the repository root instructs Git to ignore matching patterns:

```
# Ignore environment secrets
.env

# Ignore compiled artifacts and dependencies
*.log
__pycache__/
node_modules/
```

5.5 Branching and Collaborative Workflows

A **branch** in Git represents an independent line of software development. Developing on separate branches allows engineers to build features, experiment with designs, or fix bugs without destabilizing the production codebase on the **main** branch.

5.5.1 Branch Operations

- **Create and Switch to a New Branch:**

```
git switch -c feature/login-page
# Older syntax: git checkout -b feature/login-page
```

- **List Existing Branches:**

```
git branch
```

The active checked-out branch is marked with an asterisk (*).

- **Switching Between Existing Branches:**

```
git switch main
```

- **Publishing a Branch to GitHub:**

```
git push -u origin feature/login-page
```

The **-u** flag configures upstream tracking so future synchronizations require only typing **git push**.

5.6 Chapter Review & Key Takeaways

Summary Checklist

- Git is a distributed VCS created by Linus Torvalds (2005); GitHub is a cloud hosting platform for Git repositories.
- Local Git architecture flows from Working Directory $\xrightarrow{\text{git add}}$ Staging Area $\xrightarrow{\text{git commit}}$ Repository (.git).
- .gitignore prevents temporary build files, logs, and sensitive credentials from entering repository history.
- A Git branch is a lightweight movable pointer to a commit; `git switch -c <name>` creates and switches branches.
- Committing records changes locally; `git push` is required to upload commits to GitHub.

Modern AI Trends and Tools

Unit 6 Overview & Learning Objectives

This chapter explores contemporary artificial intelligence, generative models, cutting-edge software tools, prompt engineering, and ethical AI practices. By the end of this unit, students will be able to:

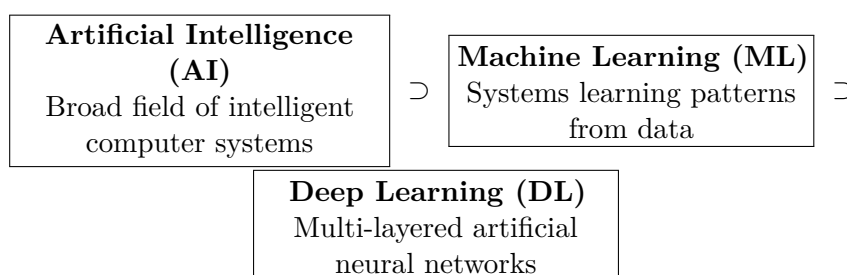
- Trace the foundations of AI from the 1956 Dartmouth Workshop to modern deep learning.
- Contrast Artificial General Intelligence (AGI) with modern Narrow (Weak) AI systems.
- Understand the mechanics of Generative AI across text, visual, and multimodal media.
- Evaluate leading AI productivity tools: ChatGPT, Google Gemini, DALL-E, Perplexity AI, and NotebookLM.
- Master the 5-pillar architecture of Prompt Engineering and implement responsible, ethical AI usage.

6.1 Foundations of Artificial Intelligence

Artificial Intelligence (AI) is the branch of computer science focused on engineering computational systems capable of performing cognitive tasks historically associated with human intelligence, such as visual perception, natural language translation, speech recognition, pattern analysis, and automated decision-making.

6.1.1 Historical Inception & The AI Hierarchy

The term “Artificial Intelligence” was formally coined in **1956** by mathematician John McCarthy and colleagues at the **Dartmouth Summer Research Project on Artificial Intelligence**.



- **Machine Learning (ML):** Instead of programmers writing hardcoded rules for every scenario, ML algorithms train on large datasets to automatically discover underlying statistical patterns (e.g., spam email filters learning word frequency correlations).
- **Deep Learning (DL):** A specialized branch of ML utilizing artificial neural networks structured in deep hierarchical layers. DL enables modern breakthroughs in autonomous driving, computer vision, and Large Language Models.
- **Training vs. Inference:**

- **Training:** The computationally intensive phase where a model analyzes billions of data examples and iteratively adjusts millions or billions of internal parameters (weights).
- **Inference:** The operational phase where a trained model applies its learned parameters to evaluate a brand-new user prompt or input image.
- **Narrow AI vs. AGI:** All existing commercial tools (Siri, ChatGPT, Gemini, Tesla Autopilot) are strictly **Narrow (Weak) AI**—engineered to excel in defined, bounded task domains. **Artificial General Intelligence (AGI)**, denoting a hypothetical machine with autonomous human-level reasoning and consciousness across all domains, does not yet exist.

6.2 Real-Life Applications of AI in Society

AI has transitioned from theoretical laboratory research into everyday infrastructure:

- **Healthcare:** Computer vision models analyze X-rays, CT scans, and MRI imagery to flag suspicious lesions for radiologist review, accelerating early disease detection.
- **Navigation & Transport:** Applications like Google Maps analyze real-time sensor and location streams to forecast traffic congestion and optimize transit routes.
- **Finance:** Banking algorithms analyze transaction velocity and purchasing geography in real time to intercept fraudulent credit card transactions.
- **E-Commerce & Streaming:** Recommender systems on Amazon, Netflix, and YouTube evaluate browsing patterns and similarities among user cohorts to suggest relevant content and products.
- **Predictive Maintenance:** Industrial sensors feed operational vibration and temperature logs to ML models to predict mechanical equipment failure prior to breakdown.

6.3 Generative Artificial Intelligence

Traditional predictive AI models classify existing data (e.g., “Is this email spam or not?”). In contrast, **Generative AI** learns underlying data distributions to **generate brand-new, original content** in response to natural-language user instructions.

6.3.1 Modalities of Generative AI

- **Text Generation (Large Language Models - LLMs):** Generates human-like paragraphs, explanations, code, and summaries by predicting the most statistically probable next **token** (word, sub-word, or punctuation symbol) based on preceding context.
- **Image Generation:** Diffusion models begin with random visual noise and iteratively denoise the image step-by-step, guided by a text prompt, into a coherent visual composition.
- **Time-Based Media (Audio & Video):** Generates synthetic voiceovers, musical tracks, or dynamic video scenes.
- **Multimodal AI:** Next-generation models (e.g., Google Gemini) natively process and understand multiple data formats simultaneously (text, code, imagery, audio, and video).

Common Pitfall: The Problem of AI Hallucination

Because LLMs operate by statistically predicting fluent linguistic sequences rather than

verifying objective ground truth, models can generate confident-sounding, grammatically flawless statements that are **completely factually false** or cite non-existent academic references. This failure mode is called **Hallucination**. Users must independently cross-verify AI factual statements with authoritative sources.

6.4 Modern AI Tools Suite

6.4.1 1. Text Generation & Conversational Assistants

- **ChatGPT (OpenAI):** Built on the GPT architecture, excels at natural dialogue, software code generation, text summarization, and logical problem-solving.
- **Google Gemini (Google):** A natively multimodal foundation model family designed to seamlessly analyze combinations of text, high-resolution images, video clips, and audio.

6.4.2 2. Visual Content Generation Tools

- **DALL-E (OpenAI) & Midjourney:** Generate high-fidelity photographic art and illustrations from descriptive prompts.
- **Adobe Firefly:** Image generation trained on licensed Adobe Stock imagery, engineered specifically for commercial safety and copyright compliance.
- **Inpainting vs. Outpainting:** **Inpainting** replaces or repairs a selected region inside an image; **Outpainting** extends the scene's visual content beyond the original boundaries.

6.4.3 3. Time-Based Media Generation

- **Synthesia:** Generates corporate video presentations featuring realistic synthetic talking human avatars from plain text scripts.
- **Runway:** Supports generative text-to-video and image-to-video transformations with cinematic camera controls. A core technical hurdle remains **temporal consistency** (ensuring clothing, faces, and lighting do not mutate unpredictably across frames).

6.4.4 4. AI-Assisted Research Platforms

- **Perplexity AI:** Functions as an AI conversational search engine, retrieving live web pages and appending numbered, clickable reference links to every claim.
- **NotebookLM (Google):** An AI research assistant that **grounds its responses strictly in the user's uploaded documents and notes**, minimizing hallucinations and providing direct passage citations.
- **JenniAI:** An academic writing copilot assisting students with research literature reviews, paraphrasing, and reference formatting.

6.5 Prompt Engineering Architecture

Prompt Engineering is the deliberate design and iterative refinement of natural-language input prompts to guide an AI system toward producing accurate, relevant, and well-formatted answers.

Definition: The 5-Pillar Structure of an Effective Prompt

1. **Role:** Assign an explicit persona ("Act as a computer science lecturer").
2. **Context:** Provide background information and target audience ("For first-year university engineering students").
3. **Task:** State the primary operational action verb clearly ("Compare compilers and interpreters").
4. **Constraints:** Specify clear boundaries on length, tone, and exclusions ("Limit your answer to 150 words; avoid jargon").
5. **Output Format:** Instruct how the response must be structured ("Present as a 2-column Markdown comparison table").

Poor Prompt (Vague)	Effective Prompt (Engineered)
"Tell me about computers."	"Act as a computing tutor. Explain the primary difference between DRAM and SRAM to a first-year student in under 80 words using a 2-column comparison table."
"Write about AI in health."	"Explain 2 concrete applications of AI in healthcare diagnostics. Limit your response to 120 words, provide one real-world hospital example for each, and end with two clinical limitations."

6.6 Responsible and Ethical AI Practices

Deploying AI systems introduces profound societal, legal, and academic responsibilities:

- **Algorithmic Bias & Fairness:** AI models learn patterns from historical human training data. If training datasets contain social stereotypes or demographic imbalances, the model will replicate and amplify unfair or discriminatory outcomes.
- **Data Privacy:** Users must never enter confidential company passwords, personal health records, or student private data into public AI services, as inputs may be retained or used for future model training.
- **Academic Integrity:** Submitting AI-generated essays, code, or homework assignments as one's own original creative work constitutes academic misconduct and plagiarism.
- **Deepfakes & Transparency:** Synthetic media manipulating real people's likenesses or voices without consent poses severe risks of fraud and defamation; synthetic media should always be transparently disclosed.
- **The Accountability Principle:** An AI tool is an assistant, not an autonomous legal entity. **The human user or deploying organization retains full moral and legal accountability for all decisions and outputs.**

6.7 Chapter Review & Key Takeaways

Summary Checklist

- AI was formally inaugurated at the 1956 Dartmouth Workshop; modern consumer tools are Narrow AI.
- Machine Learning discovers statistical patterns from data; Deep Learning uses multi-layer neural networks.
- Generative AI creates new content (text, image, audio, video); LLMs predict the next token.
- AI hallucination is the fluent fabrication of false claims; verification against real sources is essential.
- An effective prompt includes Role, Context, Task, Constraints, and Output Format.
- Ethical AI usage demands vigilance regarding data privacy, algorithmic bias, academic honesty, and human accountability.